In [ ]:
```
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.datasets import mnist
from tensorflow.keras import backend as K
import matplotlib.pyplot as plt
import numpy as np
```

In [2]:
```
#grabbing the mnist dataset
((X_train, Y_train), (X_test, Y_test)) = mnist.load_data()
X_train = X_train.reshape((X_train.shape[0], 28 * 28 * 1))
X_test = X_test.reshape((X_test.shape[0], 28 * 28 * 1))
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
```

In [3]:
```
lb = LabelBinarizer()
Y_train = lb.fit_transform(Y_train)
Y_test = lb.transform(Y_test)
```

In [4]:
```
#building the model
model = Sequential()
model.add(Dense(128, input_shape=(784,), activation="sigmoid"))
model.add(Dense(64, activation="sigmoid"))
model.add(Dense(10, activation="softmax"))
```

```
/home/hardware/.local/lib/python3.10/site-packages/keras/src/layers/core/d
ense.py:95: UserWarning: Do not pass an `input_shape`/`input_dim` argument
to a layer. When using Sequential models, prefer using an `Input(shape)` o
bject as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
E0000 00:00:1793359997.985418    8402 cuda_platform.cc:52] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

In [5]:
```
sgd = SGD(0.01)
epochs=10
model.compile(loss="categorical_crossentropy", optimizer=sgd,metrics=["ac
H = model.fit(X_train, Y_train, validation_data=(X_test, Y_test),epochs=e
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.1446 - loss: 2.2948
- val_accuracy: 0.2310 - val_loss: 2.2639
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.2995 - loss: 2.2417
- val_accuracy: 0.4022 - val_loss: 2.2139
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.4450 - loss: 2.1852
- val_accuracy: 0.5361 - val_loss: 2.1467
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.5443 - loss: 2.1068
- val_accuracy: 0.5552 - val_loss: 2.0508
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5963 - loss: 1.9939
- val_accuracy: 0.6195 - val_loss: 1.9147
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6387 - loss: 1.8395
- val_accuracy: 0.6559 - val_loss: 1.7381
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6749 - loss: 1.6532
- val_accuracy: 0.7052 - val_loss: 1.5416
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7130 - loss: 1.4605
- val_accuracy: 0.7266 - val_loss: 1.3536
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7381 - loss: 1.2858
- val_accuracy: 0.7541 - val_loss: 1.1926
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7592 - loss: 1.1406
- val_accuracy: 0.7778 - val_loss: 1.0626
```

In [6]:
```
#making the predictions
predictions = model.predict(X_test, batch_size=128)
print(classification_report(Y_test.argmax(axis=1),predictions.argmax(axis
```

```
79/79 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
              precision    recall  f1-score   support

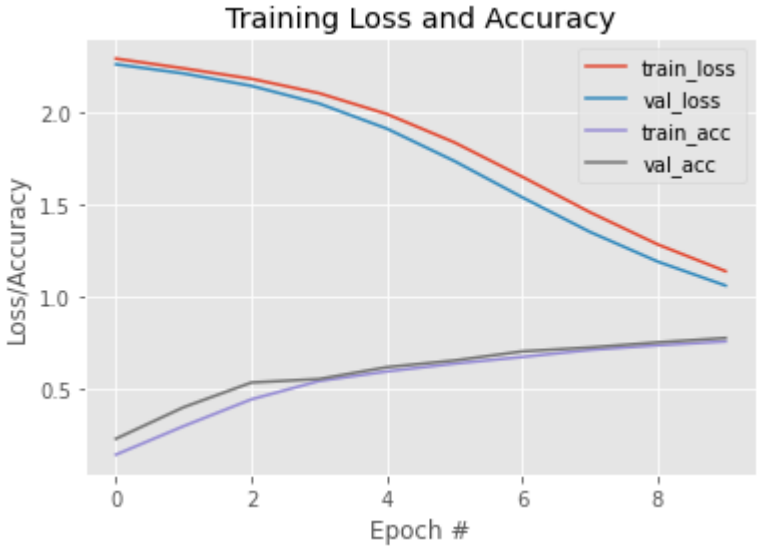
           0       0.82      0.97      0.89       980
           1       0.77      0.99      0.87      1135
           2       0.82      0.74      0.78      1032
           3       0.67      0.86      0.76      1010
           4       0.75      0.80      0.77       982
           5       0.88      0.48      0.62       892
           6       0.83      0.89      0.86       958
           7       0.73      0.88      0.80      1028
           8       0.89      0.52      0.66       974
           9       0.73      0.60      0.66      1009

    accuracy                           0.78     10000
   macro avg       0.79      0.77      0.77     10000
weighted avg       0.79      0.78      0.77     10000
```

In [7]:
```
#plotting the training loss and accuracy
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, epochs), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, epochs), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, epochs), H.history["accuracy"], label="train_acc")
```

```
plt.plot(np.arange(0, epochs), H.history["val_accuracy"], label="val_acc"
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```

Out[7]: <matplotlib.legend.Legend at 0x701344683f10>



In [ ]: